

1 Software Crisis

考试重点主要是 handbook 上的 learning outcome 中的 A-E

The challenges and problems faced in software development, particularly in the 1960s and 1970s, when demand for complex software systems outpaced the ability to design, implement, and maintain them effectively.

软件开发所面临的挑战与问题，尤其是在20世纪60至70年代，当时对复杂软件系统的需求超过了有效设计、实现和维护它们的能力。

Historical Context

The Beginning of Computing 40 ~ 50s

- Introduction to the first electronic computers, such as ENIAC and UNIVAC, which were primarily used for scientific and military applications.

第一代电子计算机（如ENIAC和UNIVAC）主要应用于科学与军事领域。

- The use of machine or assembly language, which was time-consuming and error-prone
使用机器语言或汇编语言耗时且易出错

The Beginning of Crisis 60s

- The rise of commercial computing and the need for more complex software systems, like payroll and inventory management.

商业计算的兴起以及对更复杂软件系统（如薪资和库存管理）的需求。

- Project Failures: e.g. the IBM OS/360, delays and budget overruns, the growing gap between software demand and development capabilities.

项目失败案例：例如IBM OS/360系统，项目延期与预算超支，以及软件需求与开发能力之间日益扩大的差距。

- lack of standardized methodologies and processes.
缺乏标准化的方法论与流程。

The Beginning of Software Engineering 1968-69 软件工程的开端

- Coining the Term “Software Crisis” in 1968

1968年创造“软件危机”一词

- The NATO conference acknowledged the inability to produce reliable and efficient software on time and within budget as a significant issue.

北约会议承认，无法在预算内按时开发出可靠高效的软件是一个重大问题。

- Call for a **New Discipline**: A disciplined approach to software development, laying the groundwork for software engineering.

呼吁建立新学科：以严谨方法推动软件开发，为软件工程奠定基础。

Escalation and Response 70s 升级与应对

- As systems grew more complex, the crisis deepened, projects experiencing significant overruns and failures.

随着系统日趋复杂，危机也愈发深重，各类项目普遍遭遇严重的进度延误与失败困境。

- Introduction of structured programming and the Waterfall model.

结构化编程与瀑布模型的引入。

- Recognition of the growing burden of software maintenance, which consumed a significant portion of resources and budgets.

认识到软件维护负担日益加重，其消耗了资源与预算的显著部分。

The Impact of the Crisis 危机的影响

- **Economic Consequences:** The software crisis led to substantial financial losses for companies due to failed projects and inefficient software.

经济后果：软件危机导致企业因项目失败和软件效率低下而遭受重大财务损失。

- **Human Factors:** Stress and burnout among developers became common, as they struggled to meet unrealistic deadlines and requirements.

人为因素：由于开发者们努力满足不切实际的截止期限和需求，压力与职业倦怠变得普遍。

- **Technological Stagnation:** The inability to deliver reliable software hampered technological progress and innovation in various industries.

技术停滞：无法交付可靠软件阻碍了各行业的技术进步与创新。

The Path Forward 向前的道路

- **Emergence of Software Engineering:** The crisis catalyzed the development of software engineering as a distinct discipline.

软件工程学科的兴起：这场危机催生了软件工程作为一个独立学科的发展。

- **Focus on Quality and Process:** Introduction of quality assurance practices, project management techniques, and process improvement models.

注重质量与流程：引入质量保实践、项目管理技术及流程改进模型。

- **Legacy and Lessons:** The software crisis highlighted the importance of understanding software complexity, effective communication, and the need for continuous adaptation of methodologies.

遗产与启示：软件危机凸显了理解软件复杂性、实现有效沟通以及持续调整方法论的重要性。

Key Issues 一些主要存在的问题

- Project overruns in time and budget

项目在时间和预算上的超支。

- Software that fails to meet user requirements

不符合用户需求的软件

- Poor quality and unreliable software

质量低劣且不可靠的软件

- Maintenance challenges and escalating costs

维护挑战与成本攀升

The werewolf appears 新的危机出现了

Thesis: **No Silver Bullet: Essence and Accidents of Software Engineering** 没有银弹：软件工程的本质与偶然性

- published in 1986 and has since become a seminal work in understanding the inherent challenges of software development.

该书于1986年出版，自此成为理解软件开发固有挑战的奠基性著作。

- Author Background: Frederick P. Brooks, a renowned computer scientist

作者背景：弗雷德里克·P·布鲁克斯，著名计算机科学家

Core Thesis

软件开发的复杂性由其本质决定，而非技术手段可以轻易化解。

- **No Silver Bullet:** Brooks argues that there is no single breakthrough—no “silver bullet”—that will dramatically improve software development productivity or reliability by an order of magnitude within a decade.

“银弹”的幻象 (The Illusion of a Silver Bullet)：Brooks 借用民间传说中杀死狼人的“银弹”来比喻那些被人们寄予厚望、能够瞬间解决软件危机（进度延误、预算超支、产品缺陷）的突破性技术；在十年内，不存在任何单一的技术或管理革新，能像硬件性能（摩尔定律）那样，让软件的生产力或可靠性实现**数量级（10倍以上）**的提升。

- **Essence vs. Accidents:** He distinguishes between the essential difficulties of software engineering and the accidental difficulties.

本节课最核心的讨论：

- **本质困难 (Essential Difficulties):** 软件系统固有的复杂性。包括如何逻辑严密地描述复杂的业务规则、确保成千上万个状态的一致性、以及处理不可见的概念构架。
 - 即使你有一支完美的画笔，构思一副传世名画的“灵魂”依然是极难的。
- **偶然困难 (Accidental Difficulties):** 在实现过程中引入的、非必要的困难。比如繁琐的语法、缓慢的编译速度、简陋的调试工具等。
 - 过去我们要自己研磨颜料（偶然困难），现在买现成的画具就能解决。

become a monster of missed schedules, blown budgets, flawed products ...”, in short “a werewolf” the solution to which is a “silver bullet” that “...makes software costs drop as rapidly as computer hardware costs.

成为错失期限、预算超支、产品缺陷频出的“怪物”.....简而言之，一个“狼人”，而解决之道在于找到使其“软件成本如计算机硬件成本般急速下降”的“银弹”。

Silver Bullet

- Brooks uses the "silver bullet" as a metaphor for a single technology or management technique that could magically "lay to rest" the monsters of missed schedules, blown budgets, and flawed products in software engineering.

布鲁克斯将“银弹”用作一个隐喻，意指某种单一技术或管理方法能够神奇地“平息”软件工程中进度延误、预算超支和产品缺陷这些怪物。

- He defines a true silver bullet as something that, by itself, promises at least an order-of-magnitude (tenfold) improvement in productivity, reliability, or simplicity.

他将真正的银弹定义为：其本身就能在生产力、可靠性或简洁性方面带来至少一个数量级（十倍）的改进。

Essence

- This is the inherent, irreducible "hard part" of software.
- It is the abstract "construct of interlocking concepts"—the data sets, relationships, and algorithms—and the difficult task of specifying, designing, and testing that conceptual construct.
- Brooks identifies four inherent properties of this essence that make it difficult to master: complexity, conformity, changeability, and invisibility.

这是软件固有的、不可简化的“硬核”部分。它是由相互关联的概念——**数据集、关系与算法**——构成的抽象“概念体系”，以及对该概念体系进行**规范、设计和测试**的艰巨任务。布鲁克斯指出了这一本质所固有的四个特性，使其难以驾驭，分别是：**复杂性、一致性、可变性、不可见性**。

Accident

- Difficulties that are not intrinsic to the nature of software but are instead byproducts of the methods and environments in which software is developed.
- Examples include syntax errors, slow compilation times, and manual memory management

并非软件本质固有的困难，而是**软件开发方法及环境**所产生的**衍生问题**。例如**语法错误、编译时间过长以及手动内存管理**等。

Targeting Essence or Accident ?

- Moving from Assembly to Python
从汇编到Python
- Domain-Driven Design
领域驱动设计
- Cloud Infrastructure
云服务
- Test-Driven Development
测试驱动开发

I believe the hard part of building software to be the specification, design, and testing of this conceptual construct, not the labor of representing it and testing the fidelity of the representation. [Brooks]

我认为构建软件的难点在于这个**概念结构本身的规格制定、设计与测试**，而非将其表达出来并验证表达准确性的过程。[Brooks]

Essence

Essential Difficulties 1 - Complexity 复杂性

- Vast number of states and interactions within the system.
- An essential property, not accidental
- Difficulties:
 - Communication
 - Understanding all states, unreliable
 - Extending functions without side-effect
 - Hidden states that constitute security trapdoors

Essential Difficulties 2 - Conformity 一致性

- Software must conform to human institutions and systems, which are themselves complex and ever-changing.

Software must conform because: -

- It is the most recent arrival on the scene
- Perceived as the most conformable

Essential Difficulties 3 - Changeability 可变性

- Software is subject to continuous change due to evolving requirements, technologies, and environments.
- All successful software get changed: -
 - It is being used at the edge of or beyond the original domain
 - Survives beyond the normal life of the machine it was written for

Essential Difficulties 4 - Invisibility 不可见性

- Unlike physical systems, software lacks a physical form, making it difficult to visualize and conceptualize.

Essential Difficulties – Case Studies

- Complexity
 - The "state" of the system was unintended: a mix of 7 updated servers and 1 legacy server. The developers failed to account for the **interlocking complexity** of how a single repurposed flag would behave if the environment wasn't perfectly uniform.
- Invisibility
 - Unlike a bridge where you can see a missing support beam, Knight Capital's engineers couldn't "see" that the 8th server was running old code.

- Conformity
 - Knight Capital had to change its software to conform to the NYSE's new RLP rules. This external pressure to conform often leads to "accidental" shortcuts—like repurposing an old flag rather than designing a clean, new interface.
- Changeability
 - The NYSE changed its rules for trading. If Knight Capital's system had been a hardware circuit, they might have said, "We can't change this fast enough."
 - Because it was software, there was an assumption that it *could* and *should* be changed instantly to meet the deadline. This pressure to exploit software's changeability is exactly what led to the rushed, manual deployment that skipped the 8th server.

Accident

Accidental Difficulties

- Accidental difficulties are the challenges that arise from the current state of **technology, tools**, and **practices** used in software development.

偶然性困难源自软件开发中当前**技术、工具及实践**现状所引发的挑战。
- These difficulties are not intrinsic to the nature of software but are instead **byproducts** of the methods and environments in which software is developed.

这些困难并非软件本质所固有，而是软件开发方法与环境**衍生的副产品**。

Exercise

Accidental Difficulties 是指那些由于我们选择的**开发环境、编程语言或特定实现方式**所产生的困难。它们不是问题本身固有的，而是“自找的”。

- **语言特性驱动**：段错误通常源于 C++ 的特性（如指针操作、手动 `new/delete`）。如果你换一种具有垃圾回收机制（GC）的语言（如 Java、Python 或 Go），或者使用现代 C++ 的智能指针（`std::unique_ptr`），这个问题在很大程度上会消失。
- **非业务逻辑相关**：解决段错误并不会让你离解决业务目标（例如“计算财务报表”或“渲染 3D 图形”）更进一步，它只是在修补你所使用的工具带来的副作用。

Essential Difficulties 是指软件系统**本质上**固有的复杂性。无论你用什么语言、什么工具，这些困难都无法消除。

- **逻辑复杂性**：如果你的算法逻辑本身极其复杂，或者业务规则之间存在冲突，无论你用 C++ 还是 Python，这种逻辑上的“烧脑”都是避不开的。
- **一致性与不可见性**：确保一个分布式系统在断电时数据不丢失，这就是本质困难，因为它涉及到物理现实和逻辑完备性的挑战。
- Categorize each as either an Essential Difficulty or an Accidental Difficulty, and provide a brief justification

- Determining the interlocking business rules for a cross-border tax calculation engine.

essential difficulties

- Configuring the YAML files for a Kubernetes cluster to ensure high availability.

accidental difficulties

- Resolving a "Segmentation Fault" caused by manual memory management in C++.

accidental difficulties

- Mapping out the conflicting requirements between three different stakeholders for a new hospital management system.

essential difficulties

- Optimizing the syntax of a Python script to run faster on a specific legacy processor.

accidental Difficulty

- Software complexity leads to many difficulties, what are they?

essential difficulties

lead difficulties in communication. will further cause misunderstanding.

- In your opinion, why is software perceived as the most conformable element?

essential Difficulty

- Define essential difficulties and accidental difficulties in the context of software engineering. Provide two examples of each type of difficulty.

■ **Essential Difficulties (本质困难)**

定义： 软件系统固有的、无法简化的核心复杂性。这涉及到如何将现实世界极其复杂的逻辑映射成严谨的抽象概念（数据结构、算法、交互关系）。

- **示例 1：需求规格说明 (Specification)。** 试图搞清楚银行系统如何处理几千种复杂的利息计算规则，并确保没有逻辑冲突。
- **示例 2：分布式系统的一致性 (Consistency)。** 在高并发环境下，确保全球数千台服务器上的数据状态实时、准确、无误。

■ **Accidental Difficulties (偶然困难)**

定义： 并非软件本质固有，而是由于我们使用的工具、编程语言、或特定的开发环境所产生的副作用或人为障碍。

- **示例 1：语法与样板代码 (Syntax & Boilerplate)。** 比如在 C 语言中为了实现一个简单的列表，需要写大量冗长的指针操作代码。
- **示例 2：环境配置与依赖管理 (Environment & Dependencies)。** 比如因为 Python 版本冲突或编译器设置错误导致程序无法运行。

Past Breakthroughs 过往突破

A successful past (in 1986)

- High level languages 高级语言
 - **Most important productivity development**
最重要的生产力发展
 - Reduces accidental complexity
减少意外复杂性
- Time sharing and development interactivity
分时共享与开发互动性

- **Immediacy** allows concentration
- 即时性有助于集中注意力。
- Unified programming environments

统一编程环境

- e.g. Unix, provides a workbench and tool

Lethal Silver ? 下面的那些技术可以用于根本上解决 Essential Difficulties

Brook在分析完“本质”与“偶然”困难后，逐一审视了当时（1987年前后）被人们寄予厚望、认为可能成为“银弹”的各种技术。

但是他最后得到的核心结论是：**这些技术大多只能解决“偶然困难”（Accidental Difficulties），而无法触及软件开发的“本质困难”（Essential Difficulties），因此它们都不是银弹。**

- Better High-Level Languages?
 - 像 Ada、C++ 或后来的 Python、Java 等。
 - **Brooks 的观点：**高级语言通过消除低级的语法细节（如寄存器分配、繁琐的调用约定）解决了大量的**偶然困难**。
 - **结论：**随着语言越来越高级，解决偶然困难的边际收益递减。剩下的核心逻辑复杂性（本质困难）依然存在。
- Object Oriented programming?
 - 使用类、继承和封装来组织代码。
 - **Brooks 的观点：**OOP 提高了代码的移用性（Reusability）和模块化，减少了重复造轮子的偶然困难。
 - **结论：**尽管它是一个巨大的进步，但设计一个复杂系统的“类层级结构”和“对象交互逻辑”本身依然极其困难（本质复杂性）。
- Artificial intelligence
 - 当时人们期望 AI 能自动理解需求并写出代码。
 - **Brooks 的观点：**区分了“强 AI”和“实用 AI”。他认为 AI 主要是提高生产力的工具。
 - **结论：**软件最难的部分是**定义需求**，而 AI 无法替人类决定到底想要什么（本质困难）。
- Expert systems
 - 基于规则的推理系统，模拟人类专家的决策。(such as copilot)
 - **Brooks 的观点：**专家系统可以辅助初级程序员，或者作为调试工具。
 - **结论：**它只是改进了过程，并没有简化软件逻辑结构的构建过程。
- “Automatic” programming
 - 通过高层级的需求描述自动生成可执行代码。
 - **Brooks 的观点：**他认为这本质上只是“更高级的语言”。
 - **结论：**即使代码是自动生成的，你依然需要一套极其严密、逻辑一致的“规格说明”来告诉机器生成什么，这本身就是**本质困难**。
- Graphical programming
 - 使用流程图或视觉组件代替文本编写程序。

- **Brooks 的观点：** 软件的本质是多维的、抽象的逻辑，流程图只能展示极简单的、二维的控制流。
 - **结论：** 软件的复杂性来自于隐藏的逻辑关系，而不是代码是“写”的还是“画”的。
- Program verification
 - 使用数学方法证明代码的正确性。
 - **Brooks 的观点：** 这只能证明代码符合“规格说明”，但无法证明“规格说明”本身符合“用户的真实需求”。
 - **结论：** 它能减少 Bug，但不能降低设计系统的复杂度，且成本极高。
- Environment and tools
 - 更好的 IDE、版本控制系统（Git）、自动化构建工具。
 - **Brooks 的观点：** 这些确实能让开发更顺畅，但属于对“偶然困难”的修补。
 - **结论：** 工具能提速，但不能替你思考逻辑。
- Workstations
 - 为每个程序员提供更强大的个人计算资源（当时从主机终端向个人电脑转型）。
 - **Brooks 的观点：** 减少了等待编译和运行的时间。
 - **结论：** 这纯粹是硬件层面的进步，解决了**偶然困难**，对思考软件逻辑没有直接帮助。

Lethal Silver - Discussion

- Brooks dismissed(驳回) Artificial Intelligence and Expert Systems as "Lethal Silver" in 1986.
- Today, Large Language Models (LLMs) like GitHub Copilot are often framed as a revolution
- **Does AI solve Essential or Accidental difficulties?**

AI 极大地消灭了“偶然困难”（Accidental Difficulties），但在挑战“本质困难”（Essential Difficulties）上仍处于辅助地位。

“AI 并不是杀死狼人的‘银弹’，而是一把极其锋利的‘手术刀’。”

虽然 LLM 让我们在处理“偶然困难”时实现了数量级的飞跃，使得编写代码本身变得前所未有的简单，但它并没有减轻“思考逻辑”的痛苦。

随着 AI 消灭了大部分编码工作（偶然困难），软件开发的重心将进一步向**本质困难**偏移。未来的程序员将更像“架构师”和“审查者”：我们需要花费更多精力去确保 AI 生成的逻辑符合复杂的现实业务，并处理那些 AI 无法理解的人类需求冲突。

The Promising Attacks 有希望的“攻击”

"Promising Attacks"（有希望的攻势） 是指那些虽然称不上“银弹”（不能瞬间带来 10 倍提升），但能够切实有效降低软件开发成本、管理复杂性的现实策略。

- Build vs. Buy
 - 传统的软件开发倾向于“量身定制”（Build），但这会引入大量的本质困难（需求理清、逻辑设计）和偶然困难（编码、测试、Bug 修复）。Brooks 建议尽可能选择“购买”（Buy）。
 - **不开发，直接买（Not to build, to buy）：** 如果市面上已经有成熟的解决方案，直接集成比从零开发要高效得多。
 - **大众市场的现成品（Off-the-shelf for mass market）：** 针对大众市场设计的软件（如数据库系统、云服务、办公软件）经过了数百万用户的验证。

- **即时交付 (Immediate delivery)** : 省去了漫长的开发周期。
 - **更少的错误 (Less errors)** : 成熟产品在发布前已经过严格测试, 且有大量用户反馈。
 - **低成本 (Low cost)** : 开发成本由成千上万个用户分担 (Cost distributed) , 这比单独为一个公司定制要便宜得多。
- Example:
 - API-driven development
 - 现在的开发者不需要自己写短信发送逻辑、支付接口或地图渲染引擎。
 - **本质**: 通过集成 Stripe (支付)、Twilio (通信) 或 Google Maps API, 你将这些复杂的**本质困难**外包给了专门处理这些问题的专家。
 - **结果**: 你只需要处理自己业务最核心的逻辑, 极大地缩短了开发路径。
 - Software framework
 - 如 React, Spring Boot, 或 Django。
 - **本质**: 框架定义了软件的“骨架”。它处理了如何处理 HTTP 请求、如何连接数据库、如何管理状态等通用逻辑。
 - **结果**: 框架消除了大量的**偶然困难** (如底层的网络协议处理) 和部分**本质困难** (如通用的系统构架设计) , 让开发者专注于业务创新。

Requirement Refinement & Prototyping 需求细化与原型设计

- Hardest part of building a software system is deciding precisely what to build.
构建软件系统最困难的部分在于精确决定要构建什么。
- Impossible to specify completely, precisely, and correctly the exact requirements.
无法完全、精确且正确地指定确切的需求。
- **Iteratively** deciding "precisely what to build"
迭代式确定“具体构建内容”

Incremental Development 迭代开发

- "Growing" software bit by bit rather than trying to build a complex system all at once.
逐步“培育”软件, 而非试图一次性构建复杂系统。
- **grow, don't build**
增长而不是创建

Great Designer 优秀的设计者

- Great designs come from great designers.
杰出设计源于卓越设计师。
- Best designers produce structures that are **faster, smaller, simpler, cleaner, and produced with less effort.**
优秀设计师打造的架构更快捷、更精简、更简洁、更清晰, 且投入更少。
- Great products come from one or a few designing minds, great designers.
卓越的产品源于少数设计天才的匠心独运。

Relevance Today

- **Enduring Insights:** Despite technological advancements since the article's publication, the core challenges identified by Brooks remain relevant.

持久洞见： 尽管自文章发表以来技术已取得长足进步，Brooks 所指出的核心挑战至今仍具现实意义。

- **Exercise:** Discuss how contemporary practices like Agile and AI-enhanced tools are used to tackle essential difficulties identified by Fredrick Brooks.

练习： 讨论敏捷开发与人工智能增强工具等当代实践如何应对 Brooks 所指出的本质性困难。

The Laymen Terms

- **Essential difficulties** are the hurdles that are part of the nature of software. They are "essential" because even if you had a perfect computer, these problems would still exist.

本质困难是软件固有属性中的障碍。它们之所以"本质"，是因为即便拥有完美的计算机，这些问题依然存在。

- This is the **Logic**. It is the task of deciding exactly how the tax laws should be calculated, or how 100 different microservices should talk to each other without crashing.

这就是逻辑层。它的任务是精确决定税法应如何计算，或是确保上百个微服务在相互通信时不会崩溃。

The Laymen Terms

- **Accidental difficulties** are the hurdles we encounter because of the way we build software today. They are not part of the problem itself; they are just part of our current technology

偶然性困难源于当前软件开发方式所带来的障碍。这些问题并非任务本身固有，而是现有技术局限性的体现。

- This includes things like managing memory manually (in C), dealing with slow compilers, inefficient server configuration, or struggling with a confusing IDE.

这包括手动管理内存（如使用C语言）、处理缓慢的编译器、低效的服务器配置，或是在混乱的集成开发环境中挣扎。

Essential Reading Material

No Silver Bullet: Essence and Accidents of Software Engineering, by Frederick P. Brooks